CURSO BACKEND JAVA & QUARKUS — AULA 06

Subindo uma Aplicação Backend: Containers, Pods, Portas e Nuvem

Escrever código Java em nossa máquina de desenvolvimento é apenas a primeira metade do trabalho. Para que os usuários e sistemas clientes possam utilizar nossos endpoints, a aplicação precisa ser compilada, empacotada, executada em um ambiente isolado e exposta na rede. Esta aula desmistifica todo o ciclo de infraestrutura e implantação (deployment) do backend.

1. O QUE SIGNIFICA "SUBIR UMA APLICAÇÃO"?

1.1. Do Código-Fonte ao Processo em Execução na Memória

Quando dizemos no dia a dia que vamos **"subir"** ou **"rodar"** uma aplicação, estamos nos referindo ao ato de iniciar a execução do programa no sistema operacional:

- **1. Compilação e Empacotamento:** O código Java (`.java`) é transformado em bytecode (`.class`) e agrupado em um arquivo executável (ex: `app.jar` no Quarkus/Java).
- **2. Inicialização do Processo:** O sistema operacional aloca um espaço na memória RAM e inicia um processo filho associado à Máquina Virtual Java (JVM) ou ao executável nativo do Quarkus.
- **3. Abertura do Socket de Rede:** A aplicação solicita permissão ao sistema operacional para se vincular a uma porta de comunicação da rede (ex: porta `8080`) e fica em loop contínuo aguardando requisições HTTP.

2. O QUE É DOCKER E POR QUE USAMOS CONTAINERS?

2.1. O Problema Clássico: "Na minha máquina funciona!"

Antigamente, para subir uma aplicação em um servidor remoto, era necessário instalar manualmente a versão exata do Java, das bibliotecas do sistema e configurar variáveis locais. Se a versão do Java no servidor fosse um pouco diferente da máquina do desenvolvedor, o sistema quebrava.

Conceito Docker	Analogia do Mundo Real	Definição Técnica Simplificada
Imagem Docker	Receita de Bolo / Planta de uma casa	Um arquivo estático e imutável que contém o código compilado, as bibliotecas, o Java e as instruções para rodar o sistema.
Container Docker	O Bolo pronto / A Casa construída	A instância viva e em execução da Imagem. É um ambiente totalmente isolado (um "caixote digital") rodando dentro do servidor.
Dockerfile	Manual de Instruções	O script com o passo a passo de como o Docker deve montar e gerar a Imagem do sistema.

2.2. Exemplo Prático: Dockerfile Simples do Quarkus (Fast-Jar)

Abaixo vemos as instruções para o Docker empacotar uma aplicação Quarkus:

```
# Usa uma imagem oficial do Java 17 como base
FROM eclipse-temurin:17-jre-alpine

# Define o diretório de trabalho dentro do container
WORKDIR /work/

# Copia os arquivos da aplicação compilada para dentro do container
COPY target/quarkus-app/lib/ /work/lib/
COPY target/quarkus-app/*.jar /work/
COPY target/quarkus-app/app/ /work/app/
COPY target/quarkus-app/quarkus/ /work/quarkus/

# Informa que o container utilizará a porta 8080
EXPOSE 8080

# Comando executado quando o container for iniciado
CMD ["java", "-jar", "/work/quarkus-run.jar"]
```

3. O QUE É UM POD E O ECOSSISTEMA KUBERNETES?

3.1. Por que apenas o Docker não é suficiente em Produção?

O Docker é excelente para rodar um container isolado. Porém, em grandes aplicações enterprise, temos dezenas ou centenas de containers rodando juntos. Se um container cair no meio da madrugada, quem vai reiniciar ele automaticamente? Se o tráfego de usuários triplicar na Black Friday, quem vai criar novas cópias da aplicação para aguentar a carga?

É para isso que existe o **Kubernetes (k8s)**: um orquestrador que gerencia o ciclo de vida, a saúde e o escalonamento automático de containers distribuídos em vários servidores.

3.2. Afinal, o que é um Pod?

No Kubernetes, você nunca roda um container diretamente solto. A menor unidade de execução gerenciada pelo Kubernetes é chamada de **Pod**:

- **A Analogia da Vagem de Ervilhas:** Pense no Pod como uma "vagem" protetora, e o seu container Docker como uma "ervilha" dentro dela.
- **Casulo de Execução:** O Pod é o wrapper/envelope que envolve um ou mais containers que precisam trabalhar estritamente juntos.
- **Endereçamento Único:** Cada Pod ganha um endereço IP próprio dentro da rede interna do cluster Kubernetes, além de compartilhar volumes de armazenamento entre seus containers.
- **Efemeridade e Auto-recuperação:** Pods são efêmeros (descartáveis). Se um Pod travar ou o servidor físico onde ele está quebrar, o Kubernetes destrói o Pod estragado e cria um novinho imediatamente em outro servidor sem derrubar o sistema!

4. ENDEREÇO IP, PORTAS E EXPOSIÇÃO DE REDE

4.1. Como o Tráfego da Internet Chega até o Código Java?

Para que uma chamada HTTP feita por um celular ou navegador chegue ao seu endpoint Quarkus, precisamos entender como a rede mapeia endereços e portas:

```
Endereço IP (Qual máquina/dispositivo da rede?)
   |
   └──► Exemplo: 192.168.1.10 ou 127.0.0.1 (Localhost)

Porta TCP (Qual programa/serviço específico dentro daquela máquina?)
   |
   └──► Exemplo: Porta 8080 (Quarkus), Porta 5432 (PostgreSQL), Porta 80 (HTTP padrão)
```

Conceito de Rede	Endereço / Configuração	O que significa na prática?
Localhost (Loopback)	`127.0.0.1`	Interface interna. Significa "somente esta própria máquina pode conversar comigo".
Bind em Todas as Interfaces	`0.0.0.0`	O servidor escuta requisições vindas de **qualquer** placa ou interface de rede. Necessário para containers!
Mapeamento de Porta (Docker)	`-p 8080:8080`	Conecta a porta do seu computador físico (Host) com a porta fechada lá dentro do container.

5. VARIÁVEIS DE AMBIENTE (ENVIRONMENT VARIABLES - ENV)

5.1. O que são e por que são cruciais para o Backend?

Variáveis de Ambiente são valores armazenados diretamente no Sistema Operacional onde a aplicação está rodando, e não dentro do código-fonte Java.

A regra de ouro da arquitetura de software moderna (Princípio 3 do *12-Factor App*) dita: **"O mesmo código compilado deve rodar em qualquer lugar. O que muda de um ambiente para outro são apenas as configurações injetadas via Variáveis de Ambiente!"**.

Ambiente de Execução	Sigla Comum	Para que serve este ambiente?	Exemplo de Valor de Variável (`DB_URL`)
Desenvolvimento	DEV / Local	Ambiente do computador do próprio desenvolvedor para criar funcionalidades.	`jdbc:postgresql://localhost:5432/db_dev`
Homologação / Staging	HOM / STG	Ambiente idêntico ao de produção para testes do time de QA (Garantia de Qualidade).	`jdbc:postgresql://pg-hom.empresa.com:5432/db_stg`
Produção	PRD / Prod	Ambiente real onde os clientes e usuários finais utilizam o sistema.	`jdbc:postgresql://pg-cluster-prd.internal:5432/db_prd`

5.2. Como o Quarkus lê Variáveis de Ambiente no `application.properties`

No Quarkus, podemos usar a sintaxe `${NOME_DA_VARIAVEL:valor_padrao}`. Se a variável de ambiente existir no sistema/container, o Quarkus usará o valor dela; caso contrário, usará o valor padrão fornecido após os dois pontos.

```
# Configuração flexível de Banco de Dados no Quarkus:
quarkus.datasource.jdbc.url=${DB_URL:jdbc:postgresql://localhost:5432/app_db}
quarkus.datasource.username=${DB_USER:postgres}
quarkus.datasource.password=${DB_PASSWORD:senha_dev_local}

# Configuração da URL de uma API externa de Pagamento:
pagamento.api.url=${GATEWAY_PAGAMENTO_URL:https://sandbox.pagseguro.com}
```

5.3. Por que NUNCA colocar senhas e segredos no código?

- **Segurança:** Se você escrever a senha do banco diretamente no arquivo do projeto e enviar para o GitHub, qualquer pessoa com acesso ao repositório poderá invadir o banco de produção!
- **Privacidade e Governança:** O desenvolvedor não precisa saber qual é a senha do banco de produção. O time de infraestrutura injeta as credenciais via segredos ocultos (Kubernetes Secrets / AWS Secrets Manager).

6. APLICAÇÃO RODANDO LOCAL VS. RODANDO NA NUVEM

6.1. O que é a "Nuvem" (Cloud Computing)?

Rodar uma aplicação "na Nuvem" (como AWS, Google Cloud ou Microsoft Azure) nada mais é do que alugar capacidade de processamento, memória RAM e rede em data centers gigantescos distribuídos pelo mundo, gerenciados por grandes empresas de tecnologia através de softwares de orquestração.

Aspecto	Rodando Localmente (Sua Máquina)	Rodando na Nuvem (Cluster Kubernetes / Cloud)
Recursos Computacionais	Limitados à memória RAM e CPU do seu notebook.	Escaláveis e praticamente ilimitados conforme a demanda.
Acesso à Aplicação	Apenas o seu próprio computador acessa (`localhost`).	Acessível globalmente pela internet via domínios (DNS).
Resiliência e Tolerância a Falhas	Se você desligar o notebook, a aplicação morre.	Se um servidor físico pifar, outro assume instantaneamente sem queda.
Escalabilidade (Auto-scaling)	Manual e limitada a 1 única execução por vez.	Automática: aumenta de 2 para 50 Pods em segundos durante picos de tráfego.

7. A JORNADA COMPLETA DO CÓDIGO ATÉ O POD EM PRODUÇÃO (CI/CD)

7.1. O Pipeline Automático de Implantação

Em empresas modernas, o desenvolvedor não sobe nada "na mão". Todo o processo é automatizado através de pipelines de **CI/CD (Integração Contínua e Entrega Contínua)**:

```
1. [DEV] O Desenvolvedor escreve o código Java e faz `git push` para o GitHub/GitLab.
   |
2. [CI]  Servidores de Integração baixam o código, executam testes unitários e compilam o Quarkus.
   |
3. [CD]  O pipeline constrói a Imagem Docker e publica no Registry (ex: Docker Hub / AWS ECR).
   |
4. [K8S] O Kubernetes é notificado, baixa a nova versão da Imagem e substitui os Pods antigos sem dor de cabeça!
```

7.2. Resumo para o Desenvolvedor Backend Iniciante

- Você escreve código Java desacoplado das configurações.
- Sua aplicação é compilada e guardada dentro de uma **Imagem Docker**.
- A imagem rueda isolada dentro de um **Container** envolvido em um **Pod**.
- As senhas, URLs e segredos são injetados de fora para dentro via **Variáveis de Ambiente**.
- O **Kubernetes na Nuvem** garante que o sistema fique 24/7 no ar, rápido e seguro!